

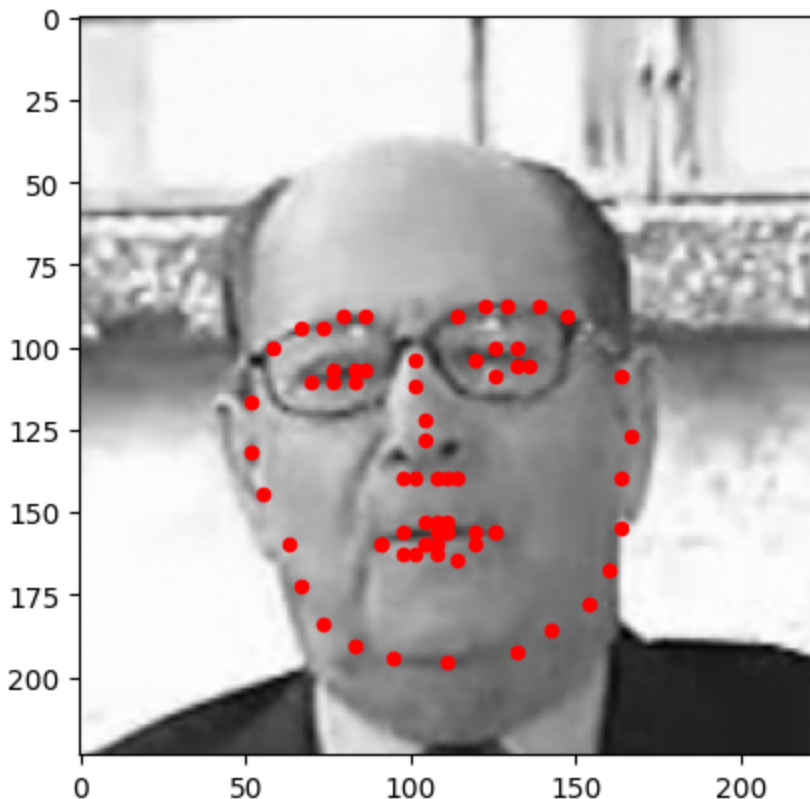
Facial Keypoint Detection

We explore 3 methods to doing facial keypoint detection with deep learning methods.

1. Direct Coordinate Regression
2. Transfer Learning
3. Heatmap prediction

We have a dataset of grayscale, 224×224 images of faces. Each image in the dataset has already been labeled with 68 keypoints, such as left eye, right eye, chin, mouth, etc. Each keypoint is an (x, y) pair. Thus we have a supervised learning objective to predict the location of new keypoints given an image of a face.

Below is an example of an image from the dataset, along with its keypoint labels:



Part 1: Direct Coordinate Regression

The first naive approach is to directly regress over the 132-dimensional label.

I used a CNN model to process features from the image, followed by a fully-connected regression head (an MLP with 2 hidden layers) to output predictions for each coordinate.

I use the architecture below:

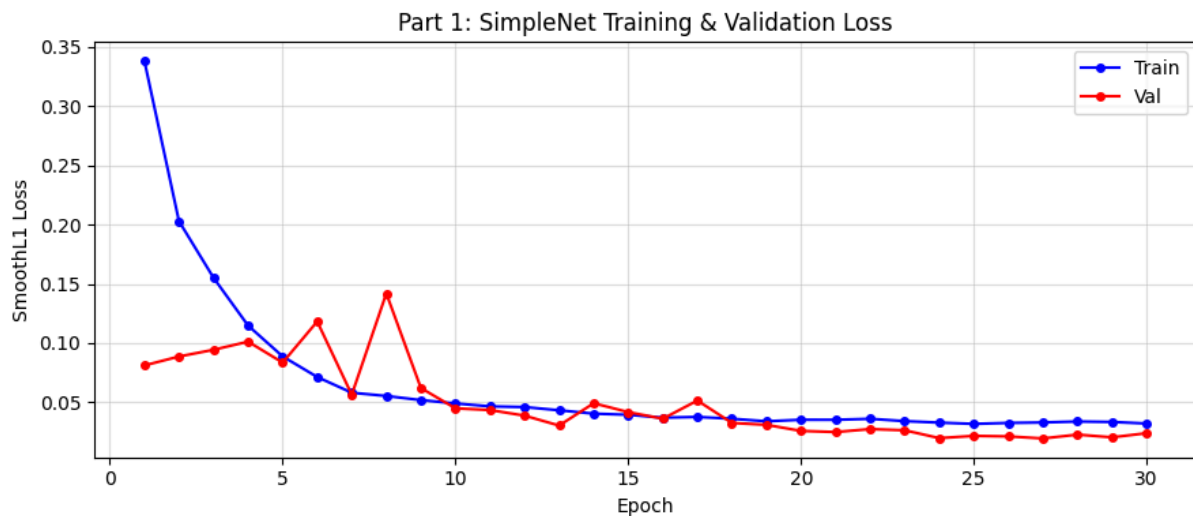
Layer (type:depth-idx)	Output Shape	Param #
SimpleNet	[1, 136]	--
└─Conv2d: 1-1	[1, 32, 224, 224]	320
└─BatchNorm2d: 1-2	[1, 32, 224, 224]	64
└─MaxPool2d: 1-3	[1, 32, 112, 112]	--
└─Dropout: 1-4	[1, 32, 112, 112]	--
└─Conv2d: 1-5	[1, 64, 112, 112]	18,496
└─BatchNorm2d: 1-6	[1, 64, 112, 112]	128
└─MaxPool2d: 1-7	[1, 64, 56, 56]	--
└─Dropout: 1-8	[1, 64, 56, 56]	--
└─Conv2d: 1-9	[1, 128, 56, 56]	73,856
└─BatchNorm2d: 1-10	[1, 128, 56, 56]	256
└─MaxPool2d: 1-11	[1, 128, 28, 28]	--
└─Dropout: 1-12	[1, 128, 28, 28]	--
└─Conv2d: 1-13	[1, 256, 28, 28]	295,168
└─BatchNorm2d: 1-14	[1, 256, 28, 28]	512
└─MaxPool2d: 1-15	[1, 256, 14, 14]	--
└─Dropout: 1-16	[1, 256, 14, 14]	--
└─Linear: 1-17	[1, 1024]	51,381,248
└─BatchNorm1d: 1-18	[1, 1024]	2,048
└─Dropout: 1-19	[1, 1024]	--
└─Linear: 1-20	[1, 512]	524,800
└─BatchNorm1d: 1-21	[1, 512]	1,024
...		
Input size (MB): 0.20		
Forward/backward pass size (MB): 48.19		
Params size (MB): 209.47		
Estimated Total Size (MB): 257.87		

along with the following hyperparameters:

- conv layers: 4
- conv kernel size: 33
- conv padding: 1 (same-padding)
- conv channels: 1 32 64 128 256
- pooling: MaxPool2d(22) after each conv
- activation: ELU
- fc layers: 50,176 1024 512 136
- weight init (FC): Xavier uniform
- dropout (conv1): 0.1
- dropout (conv2): 0.2

- dropout (conv3): 0.3
- dropout (conv4): 0.4
- dropout (fc1): 0.5
- dropout (fc2): 0.4
- loss function: SmoothL1Loss
- optimizer: Adam
- backbone LR: 4e-4
- head LR: 8e-4
- LR scheduler: StepLR
- scheduler step size: 6 epochs
- scheduler gamma: 0.5
- epochs: 30
- batch size: 16
- input size: 224x224
- rescale: 250
- random crop: 224

resulting in the following training and validation loss curves:



Let's now visualize the predicted keypoints alongside the ground truth on test images:

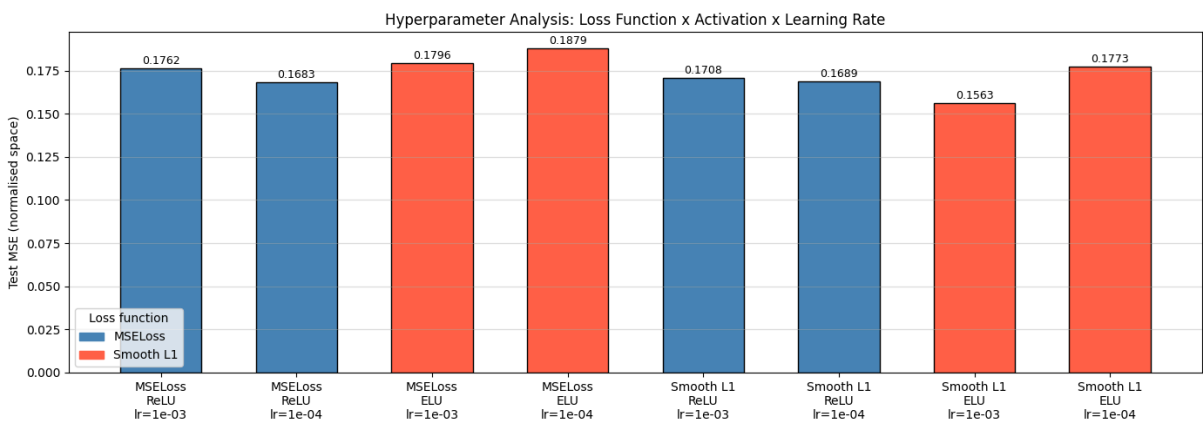


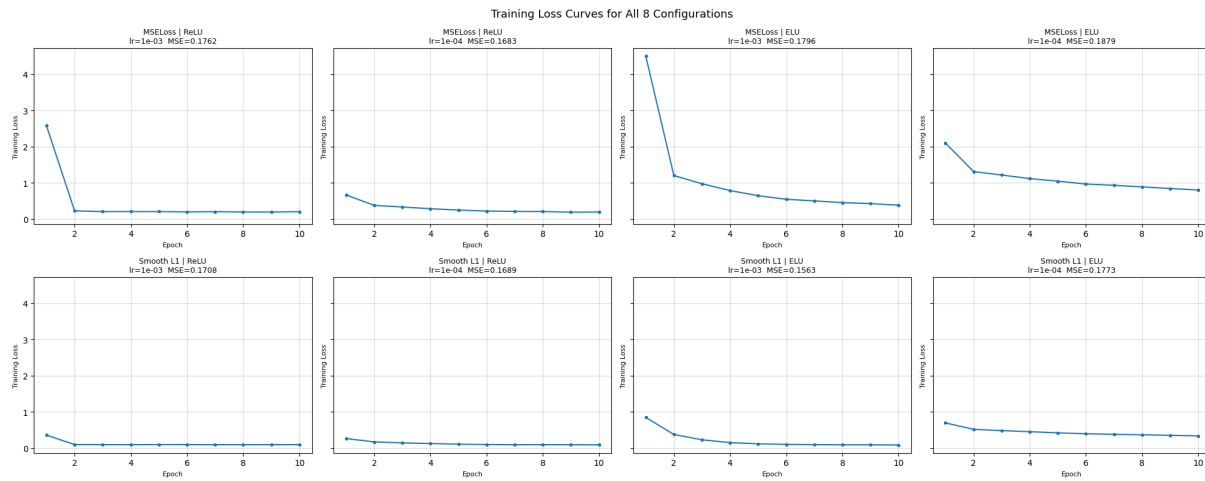
I achieved a 0.0459 MSE on the test set.

Notice that the ground truth keypoints are in green while our predictions are in red. As we see, some of the predictions are quite off. This is a challenge with regression based approaches, as the model has to learn to predict the exact coordinates of the keypoints, which can be difficult given the variability in facial features and expressions. Regression has a bias for predicting the mean of the keypoint locations, which can lead to blurry predictions that do not capture the various details of the facial features.

I performed a small hyperparameter sweep over 2 learning rates (1e-4, 1e-3), two criterions (MSELoss, SmoothL1Loss), and two activation functions (ReLU, ELU). I trained each for 5 epochs. I wished I could have done more, but my compute was extremely limited at this point. The best performing model used a learning rate of 1e-3, SmoothL1Loss, and ELU.

Here are the test set MSE losses for each model, along with the training/val loss curves for each:





Generally, the model with the higher learning rate performed better, which is expected given that the model is quite small and can easily converge to a local minimum with a low learning rate. The model with SmoothL1Loss performed better than MSELoss, which is expected given that SmoothL1Loss is less sensitive to outliers than MSELoss. The model with ELU performed better than ReLU, which is expected given that ELU can help mitigate the vanishing gradient problem and can lead to faster convergence.

Part 2: Transfer Learning

Next, we will try to leverage a pre-trained model to do the keypoint detection task. We will use a ResNet-18 model pre-trained on ImageNet as our backbone, and add a fully-connected regression head on top of it to predict the keypoints.

The model is a ResNet-18 pretrained on ImageNet, adapted for grayscale single-channel input and modified with a custom regression head.

Grayscale adaptation. ResNet-18's first convolutional layer expects 3-channel RGB input (`Conv2d(3, 64, 7x7)`). Since our images are grayscale, I replaced it with a `Conv2d(1, 64, 7x7)` layer and initialised its weights by averaging the pretrained RGB weights across the channel dimension:

```
new_conv.weight = original_conv.weight.mean(dim=1, keepdim=True)
```

This preserves the pretrained feature detectors rather than re-initialising from scratch.

Regression head. The original classification head (`Linear(512 → 1000)`) is replaced with a two-layer MLP: `Linear(512 → 256) → ReLU → Dropout(0.5) → Linear(256 →`

136) , outputting the 136 keypoint coordinates (68 keypoints $\times$ 2).

Two-phase training strategy. Training is split into two phases:

Phase 1 (frozen backbone, 15 epochs). Here, all ResNet-18 layers except `conv1` and `fc` are frozen. Only the adapted input convolution and the regression head (around 4.9M of 11.3M parameters) are trained. This lets the head learn a good mapping from the pretrained features before any backbone weights are disturbed.

Phase 2 (end-to-end fine-tuning, 20 epochs). Here, all parameters are unfrozen. A small backbone learning rate prevents destroying the pretrained representations, while a larger head learning rate continues refining the regression mapping.

Hyperparameters:

Phase 1:

- optimizer: Adam
- head LR: $1e-3$
- weight decay: $1e-4$
- scheduler: CosineAnnealingLR ($T_{\text{max}} = 15$)
- epochs: 15

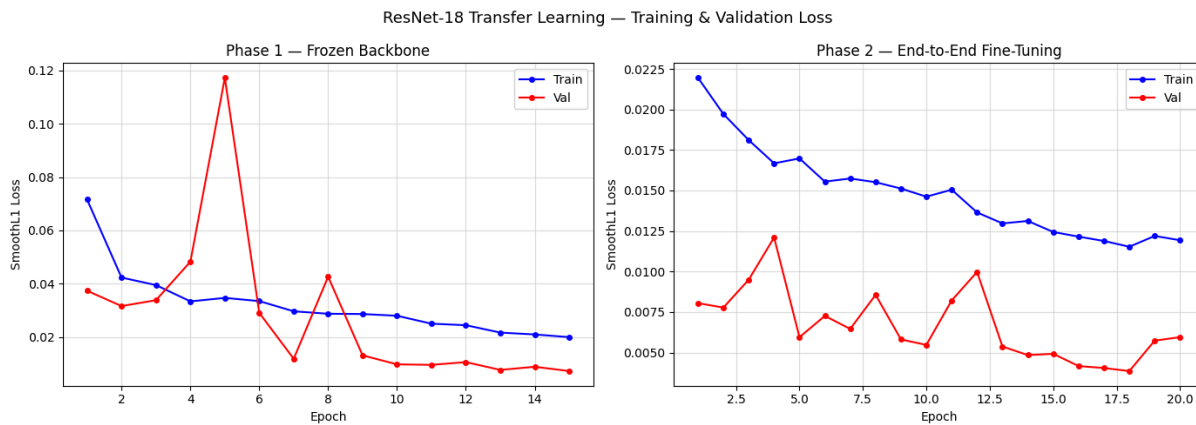
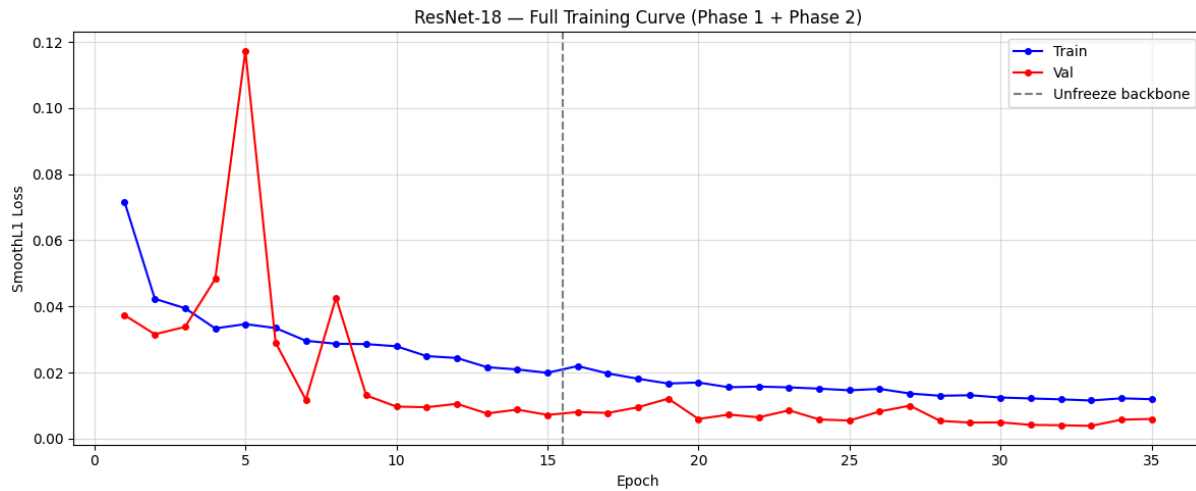
Phase 2:

- optimizer: Adam
- backbone LR: $5e-5$
- head LR: $2e-4$
- weight decay: $1e-4$
- scheduler: CosineAnnealingLR ($T_{\text{max}} = 20$)
- epochs: 20

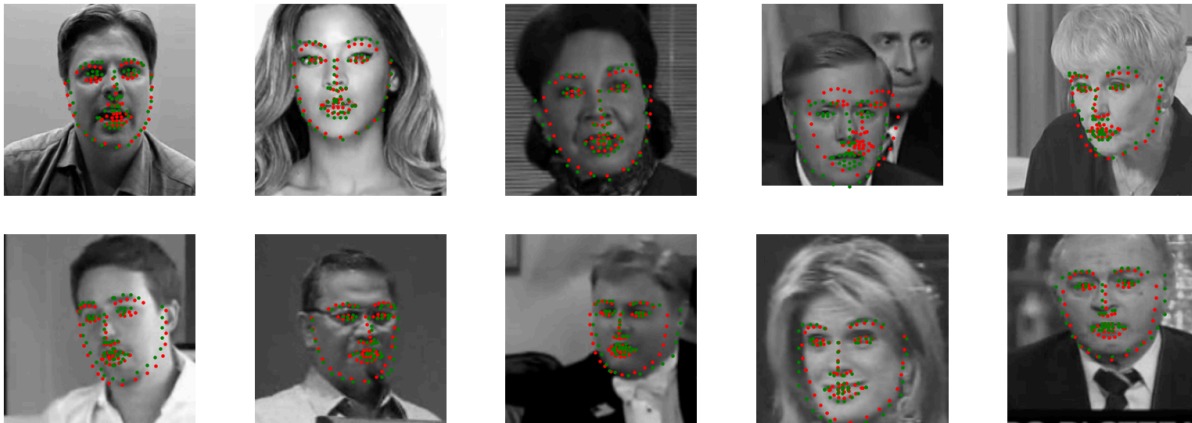
Shared:

- loss: SmoothL1Loss
- batch size: 16
- input size: 224×224

Below is the training and validation loss curves for the ResNet-18 transfer learning model:



And here are the predicted keypoints alongside the ground truth on test images:



As we see, the frequency of catastrophic failures (where the predicted keypoints are very far off from the ground truth) has decreased compared to the direct regression model, but there are still some predictions that are quite off. The MSE on the test set is 0.0076, which is a big improvement over the direct regression model. Training trajectories were also much more stable than part 1, and convergence was fast.

Part 2B: DINO Backbone

We use the same architecture and training strategy as part 2a, but replace the ResNet-18 backbone with a DINO ViT backbone. The input convolution is adapted to accept single-channel input, and the regression head is the same as in part 2a.

Model

`Dinov2_grayscale` uses a pretrained DINOv2 ViT-S/14 backbone (`dinov2_vits14` , 21M params) for keypoint regression. Grayscale input is expanded to 3 channels with `.expand()` before being passed to the backbone, which returns a 384-dim CLS token. A single linear layer maps this to 136 outputs (68 keypoints $\times$ 2 coordinates).

Training Strategy

Training follows a two-phase approach to avoid destroying pretrained features early on.

Phase 1 head only (25 epochs): The backbone is frozen and only the regression head is trained. This lets the head learn a reasonable mapping before any backbone weights are disturbed. Adam (lr=8e-4), CosineAnnealingLR, weight_decay=1e-4.

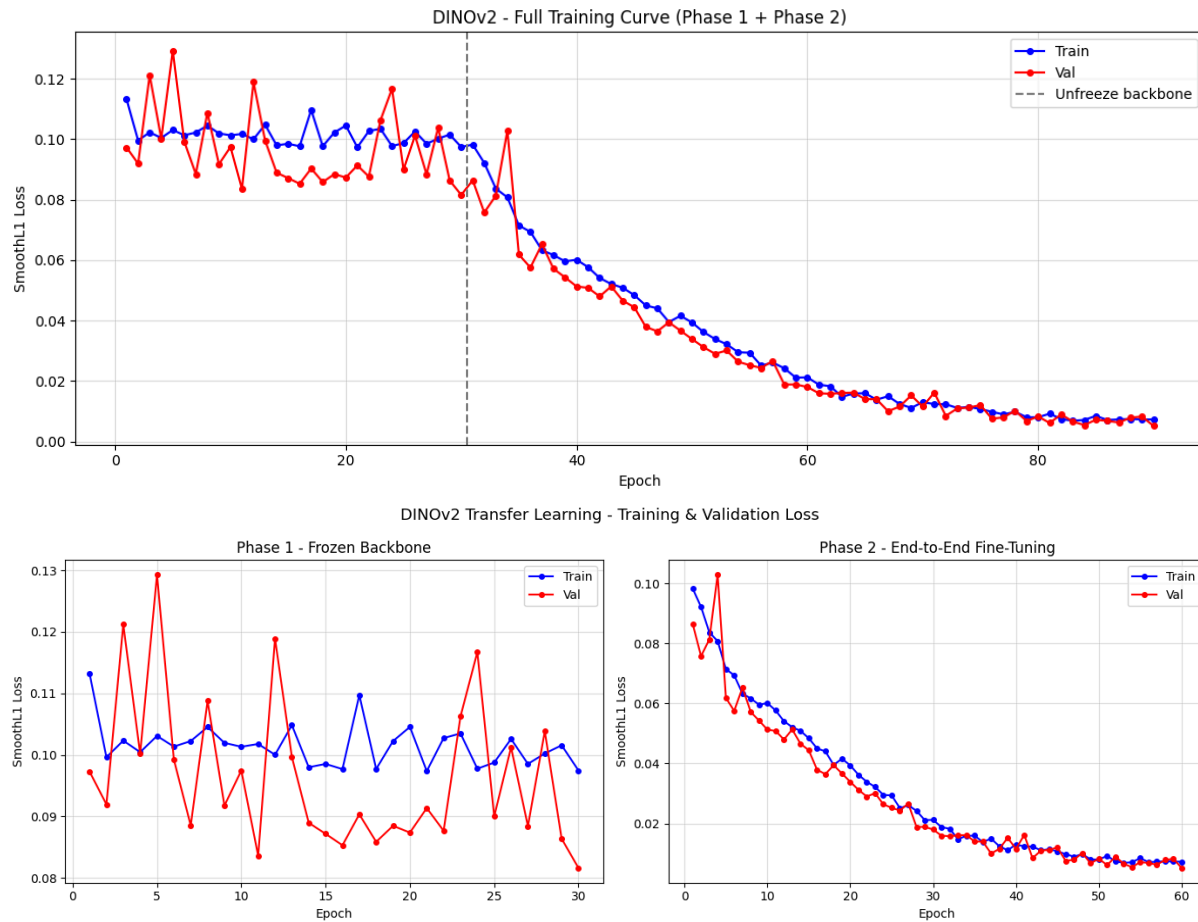
Phase 2 full fine-tuning (50 epochs): All weights are unfrozen with differential learning rates: backbone at 2e-6 (two orders of magnitude below the head) to preserve pretrained representations while allowing task-specific adaptation. Head at 4e-5. Adam, CosineAnnealingLR, weight_decay=1e-4.

Loss function: I chose SmoothL1 (Huber loss), which is less sensitive to outlier keypoints than MSE.

Hyperparameters

- Epochs: 25 (phase 1), 50 (phase 2)
- Optimizer: Adam
- Learning rates: 8e-4 (head, phase 1), 2e-6 (backbone, phase 2), 4e-5 (head, phase 2)
- Scheduler: CosineAnnealingLR
- Weight decay: 1e-4
- Loss: SmoothL1Loss

Below is the training and validation loss curves for the DINO ViT transfer learning model:



And here are the predicted keypoints alongside the ground truth on test images:



As we see, the DINO backbone generally performs better than all previous approaches, which is expected given that DINO is a much larger and more powerful model than ResNet-18, and has been trained with a self-supervised objective that encourages learning of rich visual features. The MSE on the test set is 0.0099, which is around the same as the ResNet-18 backbone. I also didn't train as many epochs as I should have due to limited compute resources. However, training was very unstable and convergence was slow (especially in the first stage), likely due to the large number of parameters in the DINO backbone and the limited size of our dataset.

Part 3: Heatmap Prediction

The final approach is to predict heatmaps for each keypoint instead of directly regressing the coordinates. The model outputs a 68-channel heatmap, where each channel corresponds to a keypoint. The predicted keypoint coordinates are obtained by taking the argmax of the heatmap for each channel.

Ground truth heatmaps are generated by placing a Gaussian blob ($\sigma=2$) at each keypoint location on a 64×64 grid, normalised to $[0, 1]$.

Architecture. I used a U-Net with skip connections. The encoder downsamples through 4 stages (each: two 3×3 convs + BN + ReLU + MaxPool), and the decoder upsamples via transposed convolutions, concatenating the matching encoder feature map at each stage. The final feature map is bilinearly upsampled to 64×64 and projected to 68 channels via a 1×1 conv.

```

Input:      (B,  1, 224, 224)

enc1:      (B,  64, 224, 224) → pool → 112×112

enc2:      (B, 128, 112, 112) → pool →  56×56

enc3:      (B, 256,  56,  56) → pool →  28×28

enc4:      (B, 512,  28,  28) → pool →  14×14

bottleneck: (B, 512,  14,  14)

up4 + enc4: (B, 512,  28,  28)

up3 + enc3: (B, 256,  56,  56)

→ upsample → 64×64 → conv → (B, 68, 64, 64)

```

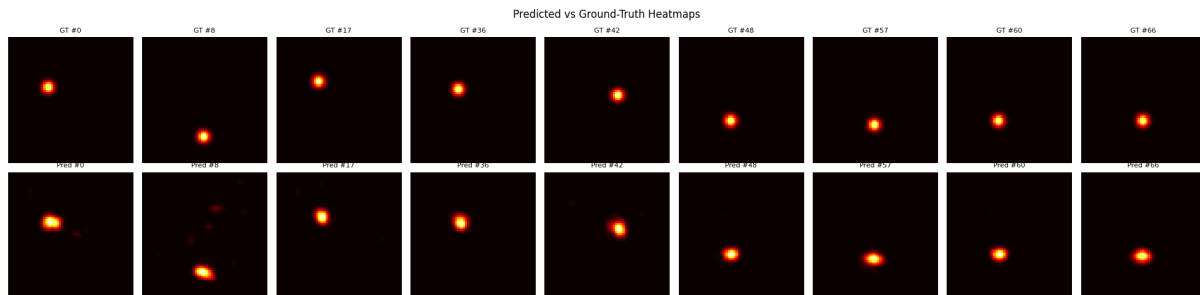
Coordinate extraction. At inference, the argmax of each heatmap channel gives the predicted pixel, which is mapped back to normalised keypoint space via $(\text{argmax} \times 224/64 - 100) / 50$.

Hyperparameters:

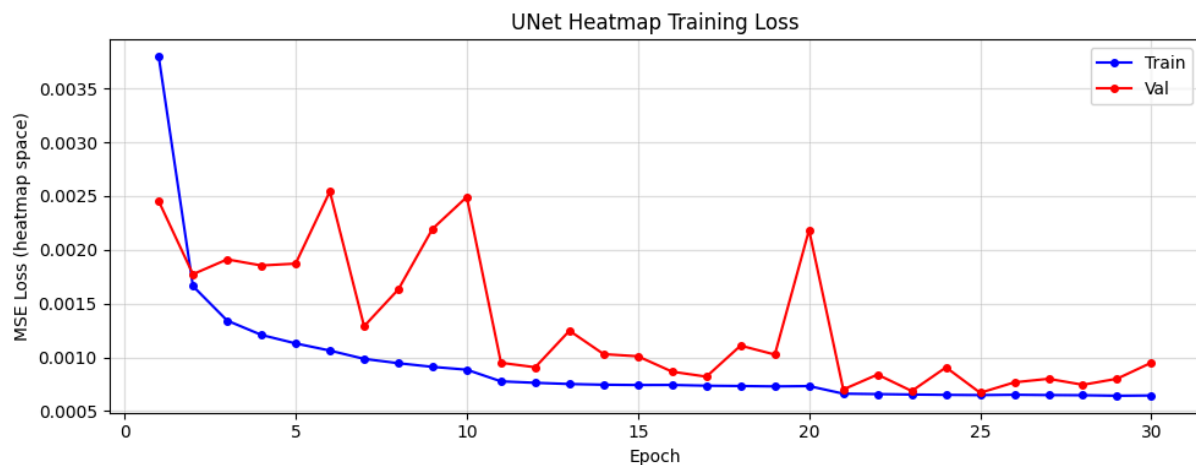
- loss: MSELoss (heatmap space)

- optimizer: Adam, lr=1e-3, weight decay=1e-4
- scheduler: StepLR (step_size=10, gamma=0.5)
- epochs: 30, batch size: 8
- heatmap size: 64×64, $\sigma=2$

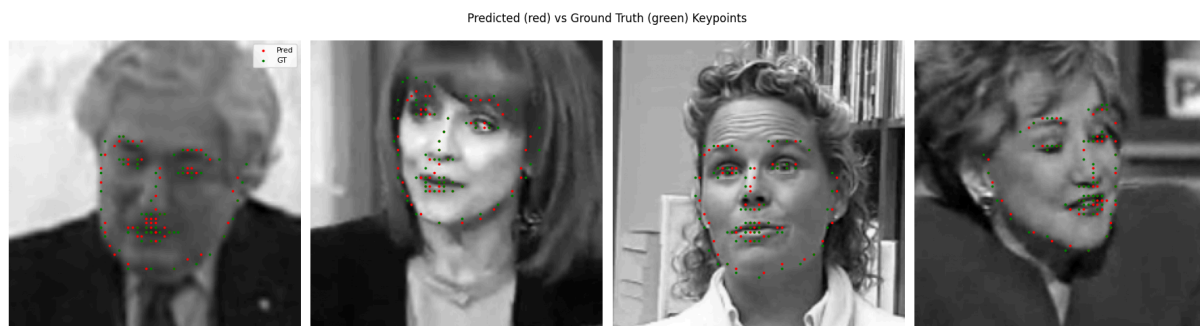
Below are the predicted heatmaps for a test image, where the bright spots indicate the model's confidence for each keypoint location:



Below is the training and validation loss curves for the heatmap prediction model::



Here are the predicted keypoints alongside the ground truth on test images:



As we see there are very few catastrophic failures and only minor errors.

My final test MSE scores for heatmap prediction was 0.0147, which was a slight downgrade from transfer learning. However, my validation MSE consistently hovered at 0.0007, which was much better than the transfer learning approach. I believe the test set performance was worse due to overfitting (or I'd just gotten unlucky), as the model had a lot of parameters and I trained for 50 epochs. With more regularization and early stopping, I believe the heatmap prediction approach could have achieved better test performance than transfer learning. So overall, the true performance of the heatmap prediction approach is the most promising out of all the methods, but further experimentation is needed to confirm this. Unfortunately, due to limited compute, I was not able to run more experiments to verify this.

Training was also remarkably stable compared to part 1, part 2a, and especially part 2b. I feel like if I ran this for more epochs with more regularization, I could have achieved even better test performance.

Conclusion

Analysis of strengths and weaknesses of each method

Overall, the direct coordinate regression approach was the simplest but had the worst performance and stability. The ResNet-18 transfer learning approach improved performance and stability by leveraging pretrained features, but still had some failures. The DINO ViT backbone had the potential for even better performance due to its powerful pretrained representations, but training was unstable and convergence was slow. The heatmap prediction approach showed the most promise in terms of stable training and good validation performance, but test performance was slightly worse than transfer learning, likely due to overfitting. With more regularization and early stopping, I believe the heatmap approach could have outperformed transfer learning on the test set as well.

The exact performances of each method were:

- Direct coordinate regression: test MSE = 0.0459
- ResNet-18 transfer learning: test MSE = 0.0076
- DINO ViT transfer learning: test MSE = 0.0099
- Heatmap prediction: test MSE = 0.0147, but validation MSE = 0.0007

Discussion of challenges faced and how you addressed them

I had quite a hard time with training stability for the DINO backbone, likely due to the large number of parameters and the limited size of our dataset. I tried various learning rates and training strategies, but ultimately the best I could do was to train for fewer epochs with a very small backbone learning rate. With more compute, I would have liked to experiment with more regularization techniques (like weight decay, dropout, data augmentation) and early stopping to see if I could achieve better performance without overfitting.

- The full code (consisting of 4 python notebooks) are in this google drive folder:
<https://drive.google.com/drive/folders/1iss4xVfDZFFIdAkCPWzwxo5X48uA4qcB?usp=sharing>
- Final model checkpoints are in the same google drive folder!